

DAA Lab - Experiment 7

Implement Dijkstra's and Bellman-Ford Algorithms for Single Source Shortest Path

Dr. Ayush Kumar Agrawal
Assistant Professor, School of Computer Science
UPES, Dehradun

Objective

- Implement and compare Dijkstra's and Bellman–Ford algorithms.
- Find shortest path from a single source to all vertices.
- Understand how they handle positive and negative edge weights.

- **Dijkstra's Algorithm:** Greedy approach; works with non-negative edge weights.
- **Bellman–Ford Algorithm:** Dynamic programming approach; handles negative weights and detects negative cycles.
- Both output shortest distances from a single source node.

Algorithm: Dijkstra's Algorithm

Dijkstra(G, source)

- ① Initialize $\text{distance}[\text{source}] = 0$, others $= \infty$
- ① While unvisited vertices exist:
 - Pick vertex u with minimum distance.
 - Mark u as visited.
 - For each neighbor v of u :
 - If $\text{dist}[v] > \text{dist}[u] + \text{weight}(u,v)$
 - Update $\text{dist}[v] = \text{dist}[u] + \text{weight}(u,v)$

Algorithm: Bellman–Ford Algorithm

BellmanFord(G , source)

- ① Initialize distance[source] = 0, others = ∞
- ② Repeat $|V| - 1$ times:
 - For each edge (u, v) :
 - If $\text{dist}[v] > \text{dist}[u] + \text{weight}(u,v)$
 - Update $\text{dist}[v] = \text{dist}[u] + \text{weight}(u,v)$
- ③ Check for negative weight cycle.

C Code - Dijkstra's Algorithm (Part 1)

```
#include <stdio.h>
#define INF 999

int main() {
    int n, source;
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int G[20][20];
    printf("Enter adjacency matrix (0 for no edge):\n");
    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++)
            scanf("%d", &G[i][j]);

    printf("Enter source vertex: ");
    scanf("%d", &source);
```

C Code - Dijkstra's Algorithm (Part 2)

```
int dist[20], visited[20] = {0};
for (int i=0; i<n; i++)
    dist[i] = (G[source][i] == 0) ? INF : G[source][i];

dist[source] = 0;
visited[source] = 1;

for (int count=1; count<n-1; count++) {
    int min = INF, u = -1;
    for (int i=0; i<n; i++)
        if (!visited[i] && dist[i] < min) {
            min = dist[i];
            u = i;
        }

    visited[u] = 1;
    for (int v=0; v<n; v++) {
        if (!visited[v] && G[u][v] && dist[u]+G[u][v] < dist[v])
            dist[v] = dist[u] + G[u][v];
    }
}
```

C Code - Dijkstra's Algorithm (Part 3)

```
printf("Shortest distances from source %d:\n", source);  
for (int i=0; i<n; i++)  
    printf("To %d = %d\n", i, dist[i]);  
  
return 0;  
}
```


C Code - Bellman-Ford Algorithm (Part 1)

```
#include <stdio.h>
#define INF 999

int main() {
    int n, e, source;
    printf("Enter number of vertices and edges: ");
    scanf("%d %d", &n, &e);

    int u[e], v[e], w[e];
    printf("Enter edges (u v w):\n");
    for (int i=0; i<e; i++)
        scanf("%d %d %d", &u[i], &v[i], &w[i]);

    printf("Enter source vertex: ");
    scanf("%d", &source);

    int dist[n];
    for (int i=0; i<n; i++) dist[i] = INF;
    dist[source] = 0;
```

C Code - Bellman-Ford Algorithm (Part 2)

```
// Relax edges |V|-1 times
for (int i=0; i<n-1; i++) {
    for (int j=0; j<e; j++) {
        if (dist[u[j]] + w[j] < dist[v[j]])
            dist[v[j]] = dist[u[j]] + w[j];
    }
}

// Check for negative cycle
for (int j=0; j<e; j++) {
    if (dist[u[j]] + w[j] < dist[v[j]]) {
        printf("Graph contains negative cycle!\n");
        return 0;
    }
}

printf("Shortest distances from source %d:\n", source);
```

C Code - Bellman-Ford Algorithm (Part 3)

```
for (int i=0; i<n; i++)  
    printf("To %d = %d\n", i, dist[i]);  
  
return 0;  
}
```

Sample Input/Output

Input (Dijkstra):

Vertices = 5, Source = 0

Adjacency Matrix:

0 10 0 5 0

0 0 1 2 0

0 0 0 0 4

0 3 9 0 2

7 0 6 0 0

Output:

Shortest distances from source 0:

To 0 = 0

To 1 = 8

To 2 = 9

To 3 = 5

To 4 = 7

- **Dijkstra's Algorithm:**

- Time: $O(V^2)$ (array), $O(E \log V)$ (heap)

- **Bellman–Ford Algorithm:**

- Time: $O(VE)$, Space: $O(V)$
- Detects negative weight cycles

Observation Table

Vertices	Dijkstra (ms)	Bellman–Ford (ms)
5	0.002	0.004
10	0.006	0.020
20	0.015	0.080

Comparison

- Dijkstra's: Faster, for non-negative edges.
- Bellman–Ford: Works for negative edges, detects cycles.
- Both produce same results for positive weights.

Viva Questions

- ① What is the difference between Dijkstra's and Bellman–Ford?
- ② Why can't Dijkstra's handle negative weights?
- ③ How does Bellman–Ford detect negative cycles?
- ④ What is the time complexity of each algorithm?

Conclusion

- Both algorithms find shortest paths from a single source.
- Dijkstra's is efficient for positive weights.
- Bellman–Ford is robust and detects negative cycles.

References

- Cormen et al., *Introduction to Algorithms*.
- Horowitz & Sahni, *Fundamentals of Computer Algorithms*.